Plan: **HymeKo-Nagare** (HymeKo-Flow) — A Paradigm-Native Dataflow ML Framework for Signed-Hypergraph Learning

2026-05-11

Etymology

HymeKo-Nagare (): *nagare* is Japanese for "flow / current / stream." Combined with the HymeKo brand and pronounced cleanly in Hungarian (*na-ga-ré*). Public-facing English alias: **HymeKo-Flow**.

Scope and motivation

PyTorch, JAX, Burn, Candle, and dfdx all share a single computational paradigm: *dense N-d tensors as the universal data type, composed into a directed-acyclic computational graph, differentiated by reverse-mode autograd over that graph.*

This paradigm is well-suited to convolutional / transformer / dense models where computation is naturally expressed as a sequence of tensor operations with a batch dimension that is the only embarrassingly parallel axis within a layer.

It is NOT a good fit for signed-hypergraph learning. Our forward computation factors as

$$F(G, X) = \sum_{c \in \mathcal{C}} f_\theta(c, X), \qquad \frac{\partial F}{\partial \theta} = \sum_{c \in \mathcal{C}} \frac{\partial f_\theta(c, X)}{\partial \theta},$$

where $\mathcal{C}$ is the cycle pool (or walk pool, or signed-hyperedge pool) and f_θ is a small per-cycle aggregator with shared parameters θ. Both forward and gradient are *sums of independent terms* over $|\mathcal{C}|$. The per-cycle computation is embarrassingly parallel; the gradient accumulation is a commutative reduction.

This is the structure of a **MapReduce / dataflow** computation, not of a tensor-op DAG. Expressing it as the latter forces:

- materialising cycle pools as $(|\mathcal{C}|, k, d)$ dense tensors (memory waste at $k \ll d$);
- per-op autograd-graph nodes that prevent operator fusion across cycles;
- serialisation through PyTorch's eager-mode Python dispatcher (the source of the 22× Rust-SIMD speedup we measured today over PyTorch eager FIR on Bitcoin OTC).

HymeKo-Nagare is the framework that takes signed-hypergraph learning's natural decomposition seriously: *operators are* (*forward, backward*) *pairs over SoA cycle pools, composed by direct function call, parallelised by cycle, executed on CPU via Rayon or GPU via wgpu/cudarc, with no autograd graph.*

Affected files

New top-level crate: `hymeko_nagare/`

```
hymeko_nagare/
 Cargo.toml
 src/
  lib.rs pub crate exports
  types.rs SoA CyclePool, Params, Grads
  ops/
   mod.rs public Op trait + registry
   fir.rs signed-cycle FIR fwd+bwd
   spline.rs Catmull-Rom signed-cycle fwd+bwd
```

```
  scatter.rs scatter-mean + scatter-sum fwd+bwd
  mlp.rs edge predictor fwd+bwd
  loss.rs BCE-with-logits fwd+bwd
  adam.rs optimizer step
 backend/
  mod.rs Backend trait
  cpu_rayon.rs Rayon parallel-by-cycle
  gpu_wgpu.rs wgpu compute-shader path
 grad_accum.rs atomic-add gradient accumulator
 training.rs forward+backward+step composition
tests/ per-op fwd+bwd unit tests
```

Touched (additive only): `hymeko_py/src/nagare.rs` (PyO3 bindings so PyTorch-using callers can drop the Rust path in during inference). PyTorch itself is untouched.

CORE.YAML items touched

None. New crate, additive throughout. Existing flat HSiKAN pipeline in `signedkan_wip/src/run_final_cell.py` stays as the training/research path. HymeKo-Nagare initially targets *inference* deployment only; training migration is a Stage 2 follow-up (see "Sequencing" below).

Mathematical foundations

Cycle-additive operator algebra

Define the cycle-additive operator class:

$$\mathcal{A} = \big\{ A : \mathcal{C} \times \mathbb{R}^{N \times d} \to \mathbb{R}^p \mid A(\mathcal{C}, X) = \sum_{c} a_\theta(c, X) \big\}.$$

Properties:

1. **Closure under sum and composition with cycle-additive aggregators.** If $A_1, A_2 \in \mathcal{A}$ and $g : \mathbb{R}^p \to \mathbb{R}^q$ is per-cycle, then $g \circ A \in \mathcal{A}$ and $A_1 + A_2 \in \mathcal{A}$.
2. **Commutativity of evaluation order.** Output is invariant under any permutation of $\mathcal{C}$ since sum is commutative. (Up to float-summation order; mitigated with Kahan summation when needed, per CLAUDE.md §5 numerical stability rules.)
3. **Gradient additivity.** For any θ, $\partial A / \partial \theta = \sum_c \partial a_\theta(c, X)/\partial \theta$. Gradient accumulation is a commutative reduction; lockless atomic-add on shared θ_{grad} is sound when the underlying float-add is atomic (`std::sync::atomic::AtomicU32` via bit-cast `f32` $\leftrightarrow$ `u32` with compare-exchange loop).
4. **Scatter-mean is a cycle-additive reduce.** The per-vertex aggregation $H[v] = \sum_{c : v \in c} a_\theta(c, X) / |\{c : v \in c\}|$ is a vertex-keyed reduction over the cycle-map outputs. Backward: each $a_\theta(c, X)$ receives gradient $\sum_{v \in c} \partial L / \partial H[v] \cdot |\{c' : v \in c'\}|^{-1}$, also cycle-additive.

Per-primitive forward + backward kernels

For each primitive we need both kernels in pure Rust:

FIR forward $\text{out}_c[j] = \sum_{i=0}^{k-1} k_{\sigma_i}[i] \cdot X[v_i][j]$ — already shipped (see `hymeko_graph::spine`).

FIR backward

$$\frac{\partial L}{\partial k_{+1}[i]} = \sum_c \mathbb{1}[\sigma_i = +1] \cdot X[v_i] \cdot \frac{\partial L}{\partial \text{out}_c}, \quad \frac{\partial L}{\partial X[v]} = \sum_{c, i : v_i = v} k_{\sigma_i}[i] \cdot \frac{\partial L}{\partial \text{out}_c}.$$

Both Rayon-parallel over cycles; the X gradient uses atomic add because multiple cycles can write to the same vertex.

Catmull-Rom spline forward Per the existing PyTorch reference in `signedkan_wip/src/splines.py`; port to pure Rust with the same numerical conventions, verified to $\leq 10^{-7}$ vs PyTorch.

Catmull-Rom spline backward Already have closed-form gradients (Triton kernel in `signedkan_wip/src/triton_fused` hands them off to the kernel); port the math to Rust.

Edge-predictor MLP forward + backward 2-layer GELU MLP; standard. ∼200 LOC including GELU + chain rule.

BCE-with-logits forward + backward

$$L = \frac{1}{N}\sum(\max(s,0) - s \cdot y + \log(1 + e^{-|s|})), \quad \partial L/\partial s = \frac{1}{N}(\sigma(s) - y).$$

Adam step

$$m \leftarrow \beta_1 m + (1-\beta_1)g, \quad v \leftarrow \beta_2 v + (1-\beta_2)g^2, \quad \theta \leftarrow \theta - \alpha\hat{m}/(\sqrt{\hat{v}} + \epsilon).$$

∼50 LOC pure Rust per parameter group.

Backend abstraction

```
trait Backend {
    /// FIR forward over a cycle pool.
    fn fir_forward(
        cycles: &CyclePool,
        features: &[f32], d: usize,
        params: &FIRParams,
    ) -> Vec<f32>;
    fn fir_backward(
        cycles: &CyclePool,
        features: &[f32], d: usize,
        params: &FIRParams,
        grad_out: &[f32],
    ) -> (Vec<f32>, FIRGrads); // (grad_features, grad_params)
    // ... same shape for spline, scatter, mlp, etc.
}

struct CpuRayonBackend;
struct WgpuBackend; // GPU via wgpu compute shaders (portable)
struct CudarcBackend; // GPU via cudarc (NVIDIA-only, faster)
```

Same operator API across backends. The execution scheduler picks a backend per-op based on input size (small graph: CPU Rayon; large graph: GPU). This is the *total parallelism* story: every backend exposes the same MapReduce primitive over cycles, and the choice is just where the map lands.

Test strategy (CLAUDE.md §3)

For each (op, backend) pair:

- **Forward parity** test against PyTorch reference to $\leq 10^{-7}$ on random fixtures (5 random shapes/inputs).
- **Numerical gradient** test: $\partial L/\partial\theta_{ij}$ via central differences matches the analytical backward to $\leq 10^{-3}$.
- **Atomic-add correctness**: 5 Rayon threads racing on the same X-gradient accumulator produce the same result as sequential evaluation (mod float-summation order, mitigated by Kahan).
- **Backend equivalence**: CPU and GPU produce the same output to $\leq 10^{-5}$ on shared random fixture.

Smoke: train an FIR-only signed-cycle model on Bitcoin OTC 1 epoch in HymeKo-Nagare, verify loss decreases monotonically, AUC > 0.55 (better than random), peak memory < 4 GB. Same smoke gate as CPML had.

Performance budget

- Bitcoin OTC inference: ≤ 1 ms / iter at $d = 32, k = 3$ (already achieved by spine alone).
- Bitcoin OTC training (FIR-only): ≤ 5 s / epoch on CPU.
- Slashdot training: ≤ 60 s / epoch on CPU; ≤ 10 s on GPU via wgpu / cudarc.
- Epinions training: ≤ 300 s / epoch on CPU; ≤ 60 s on GPU.
- Peak RAM: $\leq 4 \cdot d \cdot |V| + 4 \cdot k \cdot |\mathcal{C}|$ bytes (the SoA storage) plus $O(p)$ parameters + grads.

Sequencing (each stage is its own plan-then-implement gate)

1. **Stage 1 (inference only)**: FIR forward + scatter-mean + MLP forward + edge predictor forward, no backward, no optimizer. Drop-in inference replacement for trained HSiKAN models. CPU Rayon backend only. ∼1 month.
2. **Stage 2 (training, CPU)**: add backward for every Stage-1 op; Adam optimizer; training loop; atomic-grad-accumulator. Smoke-validate on Bitcoin OTC vs PyTorch reference. ∼2 months.
3. **Stage 3 (GPU backend)**: wgpu compute shaders for FIR fwd+bwd, scatter, MLP. Cudarc as faster NVIDIA-specific path. Backend selection at op-call time. ∼2 months.
4. **Stage 4 (full primitives)**: Catmull-Rom spline fwd+bwd in pure Rust (port from Triton); α_k multi-arity routing; attention ops if needed for SOTA parity. ∼2 months.
5. **Stage 5 (deployment)**: PyO3 bindings; ONNX-equivalent serialisation; benchmarks + paper. ∼1 month.

Total: ∼8 months for a full paradigm-native stack with PyTorch-parity training and GPU acceleration.

Rollback path

Each stage is additive. Falling back to PyTorch is always available because HSiKAN's existing PyTorch implementation stays the research path. HymeKo-Nagare is the deployment / inference / paper-systems-track companion, not a replacement during the transition.

Risk anticipation

- **Numerical drift from PyTorch**: Catmull-Rom in pure Rust may have different FP rounding than PyTorch's CUDA path; verified parity test in every op-pair test.
- **Atomic-add contention**: high-degree hub vertices receive many gradient updates from cycles incident to them. Possible contention hotspot; mitigated with per-thread accumulators combined at end of step.
- **GPU bandwidth-bound on Epinions**: scatter is bandwidth-bound on big graphs; wgpu may not match cudarc here. Acceptable: cudarc as the fast path; wgpu as portable fallback.
- **Lost research velocity**: Rust's compile-test loop is slower than PyTorch eager. Mitigate by keeping research in PyTorch (Stage 1's inference-only scope) until Stage 2 lands.
- **Ecosystem cost**: no HF, no torch.compile, no FlashAttention. Acceptable for our specific signed-hypergraph model class; revisit if scope expands to attention-heavy transformers.

Pitch

> *"Signed-hypergraph learning admits a fundamentally different computational paradigm than dense-tensor DAG autograd: cycle pools are sums of independent per-cycle terms; gradients decompose the same way. This factors into MapReduce / dataflow rather than a serial tensor-op graph, unlocking total parallelism over cycles with no batch-dimension bottleneck. We implement HymeKo-Nagare as a SoA Rust stack with per-primitive forward+backward kernels, lockless atomic gradient accumulation, and Rayon/wgpu execution backends — yielding* $22\times$ *over PyTorch eager on Bitcoin OTC and scaling linearly with core count."*

Target venue: MLSys (*The Conference on Machine Learning and Systems*), 2027 cycle. Companion: ASPLOS or PPoPP for the systems angle; ICLR if the paradigm-shift framing finds a reviewer match.